Submitted By:

Amara Hasnain (2025-CE-16)A

Submitted To:

Miss Aqsa Illiyas

Project:

"Medicine Reminder System"

For the fulfillment of,

Programming Fundamental Lab

Department of Computer Engineering

University of Engineering and Technology, Lahore.

Lab Report

Medicine Reminder System

(A Python-Based Automated Daily Medicine Scheduling Program)

1. Objective:

The objective of this lab is to design and implement a **Medicine Reminder System** using **Python**. The program helps users manage their daily medicine schedule by storing **medicine information, displaying it**, and automatically reminding the user when it is time to take a medicine. Voice alerts are also given using Windows built-in Text-to-Speech **(TTS)** feature.

2. Tools and Environment:

Function Name	Description
Language	Python
Platform	Windows (required for TTS)
IDE	Pycharm
Libraries Used	Time, Subprocess, Datetime, OS
TTS Method	Windows PowerShell - System.Speech.Synthesis
Data Storage	Plain text file (medicines.txt)

3. Theory / Background:

This program uses several basic Python concepts:

3.1 File Handling:

Python's built-in open() function is used to read and write medicine data to a **medicine.txt** file. Data is stored incomma-separated format: medicine name, time, and days.

3.2 Text-to-Speech (TTS):

Windows has a built-in speech engine called **System.Speech.Synthesis**.

We run it using Python's subprocess.Popen() by calling PowerShellcommands

directly from Python code. This avoids installing any external library.

3.3 Date and Time:

The datetime module is used to get the current time and day of the week. The program compares current time with saved medicine times to decide when to show a reminder.

3.4 Infinite Loop for Reminder:

The reminder function runs inside a while True loop and checks medicine times every **15 seconds**. When a match is found and the medicine has not been reminded already in this minute, the voice alert plays **4 times** with a **5-second gap.**

4. Program Description:

The Medicine Reminder System is a menu-based Python program. When the user runs it, they see a menu with **5** options. Below is a brief description of each function in the program:

Function Name	Description
speak(text)	Plays a voice alert using Windows TTS via PowerShell subprocess
load_medicines()	Reads medicines.txt and returns a list of medicine dictionaries
save_all(medicines)	Writes the full medicine list back to medicines.txt
add_medicine()	Takes user input for medicine name, time (HH:MM), and days, then saves it
show_medicines()	Displays all saved medicines with their time and days
delete_medicine()	Shows list and lets user delete a medicine by entering its number

check_reminders()	Loops every 15 seconds, checks current time and day, plays voice alert 4 times if match found

5. Program Menu:

Option	Action	Description
1	Add Medicine	Enter medicine name, time, and days
2	Show Medicines	Display all saved medicines
3	Delete Medicine	Remove a medicine by number
4	Start Reminder	Begin real-time reminder checking
5	Exit	Close the program

6. Source Code:

Below is the complete Python source code for the Medicine Reminder System:

```
import time
import subprocess
from datetime import datetime
import os
FILE_NAME = "medicines.txt"
#Windows built-in TTS
def speak(text):
    try:
        subprocess.Popen([
            'powershell', '-Command',
            f'Add-Type -AssemblyName System.Speech; '
            f'$s = New-Object System.Speech.Synthesis.SpeechSynthesizer; '
            f'$s.Speak("{text}")'
        ])
    except Exception as e:
print(f"Voice error: {e}")
```

```
def load_medicines():
  medicines = []
    if os.path.exists(FILE_NAME):
        with open(FILE_NAME, "r") as file:
            for line in file:
                parts = line.strip().split(",")
                if len(parts) < 3:
                    continue
                medicines.append({
                    "name": parts[0],
                    "time": parts[1],
                    "days": [d.strip() for d in parts[2:]]
                })
    return medicines

def save_all(medicines):
    with open(FILE_NAME, "w") as file:
        for med in medicines:
            file.write(f"{med['name']},{med['time']},{chr(44).join(med['days'])}
\n")

def add_medicine():
    name = input("Enter medicine name: ").strip()
    while True:
        med_time = input("Enter time (HH:MM): ").strip()
        try:
            datetime.strptime(med_time, "%H:%M")
            break
        except ValueError:
            print("Invalid format! Use HH:MM e.g. 14:30")
    days = [d.strip() for d in input("Enter days (Mon,Tue,Wed): ").split(",")]
    valid_days = {"Mon","Tue","Wed","Thu","Fri","Sat","Sun"}
    days = [d for d in days if d in valid_days]
    if not days:
        print("Invalid days!")
        return
    with open(FILE_NAME, "a") as file:
```

```
        file.write(f"{name},{med_time},{chr(44).join(days)}\n")
    print(f"Medicine '{name}' added successfully!")
```

7. Sample Output:

When the program is run, the following output appears:

```
==============================================
  Medicine Reminder System (Daily Schedule)
==============================================

1. Add Medicine
2. Show Medicines
3. Delete Medicine
4. Start Reminder
5. Exit
Enter choice: 1

Enter medicine name: Panadol
Enter time (HH:MM): 08:00
Enter days (Mon,Tue,Wed): Mon,Tue,Wed,Thu,Fri
Medicine 'Panadol' added successfully!

[When reminder fires]
REMINDER (1/4): Time to take your medicine Panadol
REMINDER (2/4): Time to take your medicine Panadol
REMINDER (3/4): Time to take your medicine Panadol
REMINDER (4/4): Time to take your medicine Panadol
Reminder complete!
```

8. Results and Discussion:

The program ran successfully and all features worked as expected:

- Medicine data was saved correctly to medicines.txt in comma-separated format.
- The show and delete functions worked properly with numbered display.
- The reminder loop checked every 15 seconds and matched the correct time and day.

- Voice alerts played 4 times with a 5-second gap using Windows PowerShell TTS.
- The reminded set prevented duplicate alerts within the same minute.

One area for future improvement is that the reminder loop runs in the main thread. If the user wants to use the menu while reminders are active, background thread (threading.Thread) could be used to run check_reminders() separately.

9. Conclusion:

This lab successfully demonstrated how to build a practical Python application using file handling, user input validation, date/time comparison, and text-to-speech integration. The Medicine Reminder System is a real-world use case that applies multiple programming concepts learned in the course. The program is simple, useful, and runs without any external Python library, making it easy to run on any Windows system.

10. References:

- Python Official Documentation - datetime module: https://docs.python.org/3/library/datetime.html
- Python Official Documentation - subprocess module: https://docs.python.org/3/library/subprocess.html
- Python Official Documentation - os module: https://docs.python.org/3/library/os.html
- Microsoft Docs - System.Speech.Synthesis Namespace: https://learn.microsoft.com/en-us/dotnet/api/system.speech.synthesis

..... THE END .....